

Build Plan

GAP 2 · GAP 3 · GAP 4 · GAP 5 · GAP 6

Architecture · Impact Analysis · Pre-Code Protocol · Failure Brainstorms

11	Items	10	Commits
4	Backend (PATH A)	6	Frontend (PATH B)
1	CRITICAL	4	HIGH
3	MEDIUM	3	LOW

Date	2026-05-13
Repository	Trading-Signals (Railway)
Indexed Symbols	712,543 symbols · 1,935,136 relationships · 300 execution flows
Status	GAP 1 COMPLETE · GAP 2 COMPLETE · GAP 3–6 PENDING
Protocol	Pre-Code Protocol · Identity Contract · Failure Brainstorm · One commit per item · Two gates

Table of Contents

1	Architecture Map	Runtime components · API surface · Dependency graph
2	Identity Contract	Golden rule · user_id context table · Query pattern
3	Non-Negotiable Gates	Six stop gates · Pre-code protocol · Commit rules
4	ITEM 1 — RangeIndex Crash	run_watch_job · line 3280 · 3-word fix
5	ITEM 2 — Null Leak in Narration	static/index.html · HOLD signals · null → plain English
6	ITEM 3 — Act Tab HOLD Direction	static/index.html · wrong direction label
7	ITEM 4 — GAP 3a: EMA Cross Invalidation	run_watch_job · watch_registry prev_ema_cross
8	ITEM 5 — GAP 3b: Supertrend Flip Invalidation	run_watch_job · watch_registry prev_supertrend_bull
9	ITEM 6 — GAP 4: Diagnostic Alert + Tighten	run_watch_job + telegram_webhook · safe_pips · new callback
10	ITEM 7a — GAP 5: News Sentiment Backend	_get_macro_context_inline · _fetch_news_sentiment · Redis cache
11	ITEM 7b — GAP 5: News Sentiment Frontend	static/index.html · signal card news row
12	ITEM 8 — GAP 6: ATR Trailing Default	AutomationSettings · _init_db · _get_automation_settings
13	ITEM 9 — Flow-Scaled Sizing Feedback Loop	static/index.html · window._userBaseRisk
14	ITEM 10 — Refresh-Logout Flicker	static/index.html · vLanding active
15	Complete Commit Sequence	All 11 commits · GitNexus risk · Path · Verification

1 · Architecture Map

GitNexus · Trading-Signals

Runtime Components

Component	Technology	Function	Deploy
Flask API	Python / Gunicorn · app.py ~10,400 lines	Signal engine · MT5 bridge · Telegram webhook · All endpoints	Railway — auto-deploy on git push
Frontend SPA	Vanilla JS / HTML · static/index.html ~12,000 lines	Dashboard · Scanner · Calculator · Act tab · Portfolio	Served by Flask /static
RQ Worker	Python / redis-queue · run_worker.py	Background jobs: auto-scan (15 min) · watch job (60s) · daily summary	Railway — second service, same codebase
PostgreSQL	Railway add-on	MT5Orders · SignalHistory · Positions · AutomationSettings · Watches	metro.proxy.rlwy.net:46116 · sslmode=disable
Redis	Railway add-on	OHLCV cache (5 min TTL) · Scanner signal cache · Dedup keys · RQ queue	metro.proxy.rlwy.net:20577
MT5 EA	MQL5	Polls /api/mt5/pending every 5s · Pushes state every 5s · Sends level alerts on TP/SL hits	Runs on trader's MT5 terminal
Telegram Bot	Telegram API webhook	Receives signals · Execute button · Close/Breakeven/Reanalyze/Tighten one-tap actions	Webhook registered on deploy

API Surface — 74 Routes (app.py)

Signal Engine	/api/analyze · /api/scan-list · /api/screen · /api/diag
MT5 Bridge (EA 5s poll)	/api/mt5/pending · /api/mt5/confirm · /api/mt5/alert · /api/mt5/push · /api/mt5/state · /api/mt5/order · /api/mt5/orders · /api/mt5/cancel/ · /api/mt5/close · /api/mt5/trailing
Automation	/api/automation/settings · /api/watch · /api/watches · /api/alert-test
Telegram	/api/telegram/webhook · /api/telegram/setup-webhook · /api/telegram-status
Portfolio / Risk	/api/positions · /api/var · /api/stress · /api/correlation · /api/optimise · /api/optimise/result · /api/signals/history
Verdict / AI	/api/verdict · /api/verdict/status · /api/verdict/result/ · /api/verdict/chat · /api/verdict/queue/clear
Market Data	/api/live-price · /api/prices · /api/econ-calendar · /api/news · /api/fear-greed · /api/sectors · /api/new-listings · /api/daily-brief
Backtest	/api/backtest · /api/backtest/enqueue · /api/backtest/result/
Pine Script	/api/pine-script · /api/pine-divergence · /api/pine-strategy
Auth	/api/login · /api/logout · /api/register · /api/auth-check · /auth/google · /auth/google/callback
Admin	/api/admin/users · /api/admin/set-role · /api/admin/invite · /api/admin/grant · /api/admin/set-tier

Dependency Graph — Key Functions (GitNexus Impact Analysis)

Function	Risk	Callers	Direct Callers	Note
safe_download	CRITICAL	10	get_mtf_trend · run_watch_job · _job_auto_scan · analyze · screen · _scan_one · get_prices · new_listings · backtest_route · _run_backtest_job	RangeIndex fix is in run_watch_job guard, NOT inside safe_download. Impact stays LOW.
_get_automation_settings	MEDIUM	5	run_watch_job · _job_auto_scan · mt5_get_pending · mt5_level_alert · automation_settings_get	GAP 6 default change must be verified against all 5 callers.
_get_macro_context_inline	LOW	1	analyze only	GAP 5 addition scoped to analyze flow only.
run_watch_job	LOW	0	Background job — no callers. Entry point only.	All GAP 3/4 changes are self-contained. Zero blast radius.

2 · Identity Contract

Golden Rule · Every Function

This contract governs every database write, every mt5_state lookup, and every query involving user ownership. It is non-negotiable and must be applied before writing any new function. Pattern 1 bugs (V, W, X, Y) in GAP 1 all resulted from violating this contract.

Context	user_id value	Examples
EA polls (/api/mt5/pending)	"default"	The EA has no session. It never authenticates as a specific user.
Telegram execute / auto-scan orders	"default"	_job_auto_scan and telegram_webhook create MT5Order(user_id="default")
Human user — frontend actions	str(session.get("user_id")) e.g. "42"	mt5_submit_order, automation_settings_save, add_watch all use session user_id
mt5_state dictionary key	EA pushes body.get("user_id", "default")	In practice always "default". Any state lookup must fallback: mt5_state.get(uid) or mt5_state.get("default", {})
Queries serving EA / background jobs	Must include "default" in filter	MT5Order.user_id.in_([uid, "default"]) — prefer human row over default row

3 · Non-Negotiable Gates

Pre-Code Protocol

Six Stop Gates — Answer every gate before touching anything. One NO = full stop.

Gate	Question	If NO
1	Was I explicitly asked to do this?	Stop. Report. Ask.
2	Can I state the symptom, root cause, and mechanism?	Investigate first. No fix yet.
3	Have I written a valid plan and got explicit confirmation?	Write plan. Wait.
4	Am I certain or assuming?	Label it. Tell user. Ask to proceed.
5	Have I stated what could go wrong?	State risks now.
6	Is this verified at runtime or only code reading?	State Level 4. Do not claim verified.

Pre-Code Protocol — Applied to every item in this document

Step 1	Write the Data Flow Specification	Before opening app.py: what data exists, where it comes from, what user_id context applies to every read and write, what fields are needed, what consumers exist downstream.
Step 2	Apply the Identity Contract	Explicitly decide for every new function: does this serve the EA ('default') or a human user (real user_id)? Write the decision in a comment at the function level.
Step 3	Map All Callers and Consumers (GitNexus)	Run gitnexus_impact() on every function being modified. Report blast radius. If impact = HIGH or CRITICAL, the plan requires user review before proceeding.
Step 4	Write the Failure Brainstorm	For every new block of logic, answer six questions: (1) data assumption gaps, (2) math edge cases, (3) empty/malformed inputs, (4) what user sees when wrong, (5) fast-clicker/race conditions, (6) cross-feature interactions.
Step 5	Write Success Criteria Before Code	Define exactly what DONE looks like with specific observable evidence, not 'the alert fires'.
Step 6	Complete Features — No Partial Propagation	If a new field is added to model A, trace every consumer of model A and update them. Never submit a field that has no reader.
Step 7	One Function, One Commit, Two Gates	Show the diff. Ask 'Shall I commit?' Wait for yes. Then ask 'Shall I push?' Wait for yes. Never bundle. Never push without explicit confirmation.

4 · Bug Fixes — Infrastructure & Frontend

ITEM 1	RangeIndex Crash — run_watch_job line 3280	CRITICAL	PATH A	1 commit
--------	--	----------	--------	----------

File	app.py — run_watch_job function (lines 3257–3662)
GitNexus risk	LOW — 0 upstream callers. Self-contained background job.
Identity contract	run_watch_job serves EA context — user_id="default". Fix touches no user_id logic.
Root cause	safe_download (CRITICAL — 10 callers) returns empty pd.DataFrame() with default RangeIndex when Yahoo Finance 429 + Binance 451. Line 3280 calls .resample('4h') without not df.empty guard → ValueError.
Affected entries	legacy_GBPUSD=X_4h · 1_ETH-USD_4h · anon_BTC-USD_4h · legacy_NZDUSD=X_4h · legacy_USDJPY=X_4h
The change	if "resample" in cfg: → if "resample" in cfg and not df.empty: (3 words added, nothing else)

Failure Brainstorm

1	df is None	safe_download returns pd.DataFrame(), not None. not df.empty on empty DataFrame = True. not None raises AttributeError — not applicable.
2	Legitimate errors suppressed	No. If df has data but wrong index type, resample still raises. Guard only skips empty data — correct behavior.
3	Downstream crash after skip	Existing len(df) < 51 bar floor check would have caught it two lines later. This fix makes the skip explicit and earlier.
4	Other resample() calls unguarded	Must grep run_watch_job for all .resample() calls and verify each has the same guard pattern.

Success criteria: No RangeIndex errors in Railway deploy logs for 5+ minutes after push. Watch entries for GBPUSD=X, ETH-USD, BTC-USD, NZDUSD=X, USDJPY=X continue silently (data unavailable = skip, not crash).

ITEM 2	Null Leak in Narration Text — HOLD Signals	HIGH	PATH B	1 commit
--------	--	------	--------	----------

File	static/index.html — narration template + Act tab WHAT WILL HAPPEN section
GitNexus risk	N/A — frontend fix. No shared CSS. Scoped to narration template strings.
Identity contract	Frontend only. No user_id context.
Root cause	HOLD signal → backend returns entry=None, stop_loss=None, tp1/tp2/tp3=None → JSON null → JS narration template interpolates \${d.entry} as literal string 'null'. Affects both WHAT THIS SIGNAL MEANS and Act tab WHAT WILL HAPPEN.
The change	Add null guards to every d.entry, d.stop_loss, d.tp1/tp2/tp3 reference in narration template. When null: replace sentence with plain-English HOLD explanation per founding principle.

Failure Brainstorm

1	Act tab separate path	Act tab WHAT WILL HAPPEN may be a different code path from signal card narration. Verify both locations independently before commit.
2	tp2/tp3 null on BUY/SELL	Some signals may have null tp2/tp3 even when not HOLD (single-TP signals). Guard must be null-specific, not signal-type-specific.
3	Over-suppression	Null guard must not suppress valid \$0 values. Check: null vs 0 vs undefined — only null/undefined should trigger fallback text.
4	CSS side effects	Narration is inline text. No shared CSS impacted.

Success criteria: Analyze any HOLD signal. Narration shows plain-English 'no trade' explanation. Zero occurrences of the word 'null' anywhere in the signal card or Act tab.

ITEM 3	Act Tab Wrong Direction Label — HOLD Signals	HIGH	PATH B	1 commit
--------	--	------	--------	----------

File	static/index.html — Act tab direction label rendering
GitNexus risk	N/A — frontend fix. Scoped to Act tab direction label.

Root cause	Act tab renders 'DotVerse will sell BTC' from a direction variable. HOLD signal has no direction — variable is stale from prior signal or defaults to SELL. Label renders incorrect direction.
The change	Gate the direction label on d.signal !== 'HOLD'. When HOLD: show 'No active direction — waiting for setup' or hide direction label entirely.

Failure Brainstorm		
1	Direction label elsewhere	Check if 'DotVerse will BUY/SELL' text appears anywhere else in Act tab beyond the identified label. Fix must be comprehensive.
2	BUY/SELL signals broken	Gate must be signal-conditional (only suppress on HOLD), not a blanket removal. Verify BUY/SELL direction still displays correctly.

Success criteria: HOLD signal → Act tab shows no direction claim. BUY signal → Act tab correctly shows 'DotVerse will buy [ticker]'.

5 · GAP 3 & GAP 4 — Signal Invalidation Enhancements

HIGH PRIORITY · Chapter 9

ITEM 4 GAP 3a — EMA Cross Invalidation Trigger

HIGH

PATH A

1 commit

File	app.py — run_watch_job only (+~15 lines in watch_registry update block + invalidation check)
GitNexus risk	LOW — 0 upstream callers on run_watch_job. Self-contained.
Identity contract	run_watch_job. MT5Order queries already use user_id.in_([uid, "default"]) from GAP 1.
PDF spec	Chapter 9: EMA fast crossing below EMA slow on a BUY trade = invalidation trigger. Mirror for SELL.

Data Flow

Data Item	Source	Available?
ind['ema_fast'] current bar	calculate_indicators output in run_watch_job	YES
ind['ema_slow'] current bar	same ind dict	YES
EMA fast previous bar	Must survive between 60s ticks	NO — add watch_registry[key]['prev_ema_cross']
order_type (BUY/SELL)	MT5Order queried by ticket	YES — already in invalidation block
Redis dedup key	"ema_cross:{ticket}:{date}" TTL=86400s	NEW

Trigger logic: BUY invalidation: prev_ema_cross == "above" AND ind["ema_fast"] < ind["ema_slow"]. SELL invalidation: prev_ema_cross == "below" AND ind["ema_fast"] > ind["ema_slow"].

Failure Brainstorm

1	prev_ema_cross missing on first tick	Default to current state on first tick — no cross can be detected. Set on first tick, check from second tick onward.
2	ema_fast or ema_slow is NaN	Guard with pd.isna() check before writing prev_ema_cross. If NaN, skip both the store and the check.
3	Same position triggers both EMA cross AND confluence	Both can fire independently. Separate Redis dedup keys. Correct behavior.
4	Redis down	Dedup skipped. Watch job continues without alerting. Acceptable — same pattern as GAP 2.
5	Watch entry removed mid-tick	watch_registry.get(key) may return None. Existing code already handles with 'if key not in watch_registry: continue'.

Success criteria: watch_registry[key] contains prev_ema_cross after first tick. On simulated EMA cross (force ema_fast < ema_slow after prev_ema_cross == 'above' on a BUY), Telegram receives one invalidation alert with EMA cross mentioned.

ITEM 5 GAP 3b — Supertrend Flip Invalidation Trigger

HIGH

PATH A

1 commit

File	app.py — run_watch_job only (+~10 lines in watch_registry update block + invalidation check)
GitNexus risk	LOW — 0 upstream callers. Self-contained. Separate commit from GAP 3a.
PDF spec	Chapter 9: Supertrend flipping direction mid-trade = high-reliability reversal signal. Must fire independently from EMA cross.
State storage	Add watch_registry[key]['prev_supertrend_bull'] = bool. Set after each tick.
Redis dedup key	"st_flip:{ticket}:{date}" TTL=86400s — separate from ema_cross key

Failure Brainstorm

1	supertrend_bull absent from ind dict	Guard with ind.get("supertrend_bull") — if None, skip both store and check. Some asset classes may not compute Supertrend.
2	First-tick initialisation	Same pattern as EMA cross — set on first tick, check from second tick.
3	Both EMA cross and Supertrend fire simultaneously	Correct. Two separate dedup keys ensure one alert per trigger type per day per ticket.

Success criteria: watch_registry[key] stores prev_supertrend_bull. Supertrend flip on a live BUY position triggers Telegram alert separate from EMA cross alert.

Files	app.py — run_watch_job alert text block + telegram_webhook new elif handler (~10 lines)
GitNexus risk	LOW on both functions — 0 upstream callers on each. Self-contained.
Identity contract	New MT5Order in tighten handler: user_id="default" (Telegram action).
Current text (generic)	"Consider moving your stop to breakeven or exiting to protect capital."
Required text (PDF Ch 9)	"Tighten stop to +{safe_pips} pips (above entry {open_price}, locking {locked_profit:.2f} profit). If confluence does not recover above 60% on next 2 candles, consider closing."
safe_pips formula	profit_pips = abs(current_price - open_price) / pip_size. safe_pips = max(1, round(profit_pips - 3, 0)). Cap: min(safe_pips, tp1_distance_pips * 0.9). new_sl_price = open_price + (safe_pips * pip_size) for BUY.
New callback_data	"tighten{ticket}{{symbol}}{new_sl_price}" — new elif in telegram_webhook (~10 lines). Creates MT5Order MODIFY with sl=new_sl_price.

Failure Brainstorm

1	open_price is None	Guard — if open_price is None, fall back to generic text only. Do not compute safe_pips.
2	profit_pips is negative (position in loss)	If profit_pips <= 3, use generic text only — no point tightening below entry.
3	tp1 not in DB for this ticket	tp1_distance_pips cap cannot be computed. Use uncapped safe_pips with a maximum of 50 pips.
4	pip_size wrong for JPY pairs	_atr_to_pips already handles JPY. Use the same pip_size logic. Do NOT compute pip_size independently.
5	tighten callback with malformed sl_price	float(sl_price) raises ValueError. Guard with try/except, answer callback with 'Invalid level' if it fails.
6	new_sl_price below entry on BUY (rounding)	Final guard: new_sl_price >= open_price for BUY, else fallback to generic text.

Success criteria: Trigger invalidation alert on a BUY position in profit. Telegram shows specific pip count ('Tighten Stop to +8 pips'), not generic text. Tap button → MT5Order MODIFY created in DB → EA picks up on next 5s poll.

6 · GAP 5 — Finnhub News Sentiment

MEDIUM PRIORITY · Chapter 4

ITEM 7 GAP 5a — News Sentiment Backend MEDIUM PATH A 1 commit

File	app.py — new <code>_fetch_news_sentiment()</code> function + addition to <code>_get_macro_context_inline</code>
GitNexus risk	LOW — <code>_get_macro_context_inline</code> has 1 caller (analyze). Adding a new key to return dict is safe.
Identity contract	No <code>user_id</code> context. Market data function — no DB writes.
New function	<code>_fetch_news_sentiment(ticker, asset_type) → dict</code> . Calls Finnhub <code>/company-news (stocks)</code> or <code>/news?category=general (crypto/forex)</code> .
Ticker mapping	Crypto: BTC-USD → BTCUSD via <code>_mt5_symbol()</code> . Forex: EURUSD=X → EURUSD.
Redis cache	"news:{ticker}:{YYYY-MM-DD-HH}" TTL=3600s. Second signal for same ticker in same hour hits cache.
Return dict	<code>{"positive": N, "negative": N, "neutral": N, "verdict": "BULLISH PRESS" "NEGATIVE PRESS" "NEUTRAL", "top_headline": "...80 chars..."}</code>
Sentiment fallback	If Finnhub article has no sentiment field: keyword scoring on headline text. Bearish: crash/drop/fear. Bullish: surge/rally/breakout.
No API key	Return None silently. Frontend hides widget. No crash anywhere.

Failure Brainstorm

1	Rate limit (60 calls/min free tier)	Redis cache per ticker per hour. Second signal for same ticker in <1 hour hits cache — no API call.
2	Empty article array	<code>verdict = "NEWS UNAVAILABLE"</code> . Grey indicator on card.
3	<code>_get_macro_context_inline</code> disruption	News call added AFTER existing CoinMarketCal and Finnhub calendar calls. Not interleaved. Function is 189 lines (1919-2108) — must not touch existing calendar logic.
4	Finnhub returns 429	Treat as empty. Cache the empty result to prevent retry storm. Log warning.

Success criteria: `response_data` contains 'news_sentiment' key after analyze. Same ticker analyzed twice in <1 hour: second call hits Redis cache (confirm via Railway logs showing no Finnhub API call on second request).

ITEM 8 GAP 5b — News Sentiment Frontend MEDIUM PATH B 1 commit

File	static/index.html — signal card event-context block
GitNexus risk	N/A — frontend fix. New row added to existing event-context section.
The change	Add news sentiment row below macro context block. Shows: green/red/grey headline counts + verdict tag + top headline in italic. Only renders when <code>d.news_sentiment</code> is non-null.
Null guard	<code>if (!d.news_sentiment)</code> return — entire row hidden when backend returns null (no API key configured).

Success criteria: Analyze BTC or EURUSD. Signal card shows news sentiment row with headline counts and verdict tag. Analyze when no Finnhub key set: news row absent, no JS error in console.

7 · GAP 6 — ATR Trailing Default 2.0x → 1.0x

MEDIUM PRIORITY · Chapter 8

ITEM 9

GAP 6 — ATR Trailing Default

MEDIUM

PATH A

1 commit

Files	app.py — AutomationSettings model + _init_db migration + _get_automation_settings fallback
GitNexus risk	MEDIUM — _get_automation_settings has 5 callers (run_watch_job · _job_auto_scan · mt5_get_pending · mt5_level_alert · automation_settings_get). Default change only affects NEW rows and fallback value — existing DB rows unaffected.
Identity contract	AutomationSettings rows created by human users (real user_id) and by EA path ('default'). No change to query logic.
PDF spec (Ch 8)	Trail distance = 1.0x current ATR. Conservative = 0.75x. Current code: trailing_atr_mult=2.0 — spec deviation.
Impact of 2.0x vs 1.0x	BTC ATR 480 pips @ 2.0x: trail 960 pips. @ 1.0x: trail 480 pips. Positions give back 2x intended profit before closing at current default.

Three Locations — One Commit

Location	Change
AutomationSettings model	trailing_atr_mult = Column(Float, default=1.0)
_init_db migration	ALTER TABLE automation_settings ALTER COLUMN trailing_atr_mult SET DEFAULT 1.0
_get_automation_settings fallback	"trailing_atr_mult": getattr(s, "trailing_atr_mult", 1.0) or 1.0

Failure Brainstorm

1	Existing DB rows already have 2.0	ALTER changes column default for new inserts only. Existing rows unaffected — by design. Verified by querying existing row after migration.
2	getattr fallback for 0.0 value	getattr(s, 'trailing_atr_mult', 1.0) or 1.0 — the 'or 1.0' handles column returning 0.0 (falsy). Shouldn't occur but safe.
3	All 5 callers receive the value	They already read it correctly. No change to their code needed. Impact is default value only.
4	run_worker.py imports AutomationSettings	Model Column default change does not change the class interface — import is safe.

Success criteria: Fresh user (new AutomationSettings row) gets trailing_atr_mult = 1.0 from /api/automation/settings. Existing user row still shows their saved value (unmodified).

8 · Frontend Residuals

ITEM 10

Flow-Scaled Sizing Feedback Loop

MEDIUM

PATH B

1 commit

File	static/index.html — updateFlowBadge + szRisk oninput handler + APPLY button
Root cause	window._userBaseRisk never initialized. updateFlowBadge reads szRisk input value. APPLY button writes to szRisk. Each APPLY → updateFlowBadge re-fires → reads reduced value → suggests further reduction → loop.
The change (Option 1)	On page load: window._userBaseRisk = parseFloat(szRisk?.value) 1.0. szRisk oninput: window._userBaseRisk = parseFloat(this.value) 1.0. updateFlowBadge: read window._userBaseRisk instead of szRisk.value. APPLY writes to szRisk only.
Pre-verify before coding	Confirm in DevTools: does programmatic inp.value = X fire oninput? If yes, need window._isApplyingFlow flag. If no, no flag needed. Verify FIRST.

Failure Brainstorm

1	Programmatic .value = fires oninput	If yes: add window._isApplyingFlow = true before APPLY writes, check flag in oninput handler, set false after. Must be verified in browser DevTools before writing any code.
2	szRisk not present on page load	Optional chain (?.) handles null — falls back to 1.0 default.
3	User clears szRisk field	parseFloat("") = NaN → 1.0 fallback correctly resets to 1.0.

Success criteria: Set szRisk to 5%. Click APPLY. szRisk shows flow-scaled value. Click APPLY again — value does not drop further. window._userBaseRisk remains 5 throughout.

ITEM 11

Refresh-Logout UI Flicker

LOW

PATH B

1 commit

File	static/index.html line 4046
Root cause	vLanding has active class by default. Auth check is async. Sign-in view renders first, then switches to vDash when fetch resolves. Appears as a logout flash on every hard refresh.
Change 1	Remove 'active' from
Change 2	In _bootAuthCheck IIFE else-branch (unauthenticated): add showView('vLanding')

Failure Brainstorm

1	showView handles vLanding	Confirm showView() exists and accepts 'vLanding' as a valid argument. Grep for all showView calls to confirm.
2	Other paths that show vLanding	Check for any other code path that calls showView('vLanding') — confirm they still work without the default active class.

Success criteria: Hard refresh while logged in → no sign-in flash. Hard refresh while logged out → sign-in view appears immediately without flash.

9 · Complete Commit Sequence

11 commits · Sequential · No bundling

Each item is one commit. No item starts until the previous is runtime-verified by the user in a live browser. Show diff → 'Shall I commit?' → wait for yes → 'Shall I push?' → wait for yes. Two gates per commit.

#	Item	Functions Changed	Risk	Path	Commits	Verification
1	RangeIndex fix line 3280	<i>run_watch_job</i>	LOW	A	1	No RangeIndex errors in Railway logs for 5+ min
2	Null leak narration	<i>index.html narration template</i>	—	B	1	HOLD signal: zero 'null' in signal card + Act tab
3	HOLD direction Act tab	<i>index.html Act tab direction label</i>	—	B	1	HOLD: no direction claim. BUY: correct label.
4	GAP 3a EMA cross	<i>run_watch_job</i> <i>watch_registry + invalidation block</i>	LOW	A	1	watch_registry has prev_ema_cross. EMA cross fires Telegram.
5	GAP 3b Supertrend flip	<i>run_watch_job</i> <i>watch_registry + invalidation block</i>	LOW	A	1	watch_registry has prev_supertrend_bull. Flip fires Telegram.
6	GAP 4 Diagnostic + tighten	<i>run_watch_job alert text + telegram_webhook elif</i>	LOW	A	1	Specific pip count in alert. Tighten tap → MODIFY in DB.
7a	GAP 5 news backend	<i>_get_macro_context_inline + _fetch_news_sentiment (new)</i>	LOW	A	1	response_data has news_sentiment. Redis cache confirmed.
7b	GAP 5 news frontend	<i>index.html signal card event-context block</i>	—	B	1	News row shows on signal card. Hidden when no API key.
8	GAP 6 ATR trailing	<i>AutomationSettings + _init_db + _get_automation_settings</i>	MEDIUM	A	1	New row gets 1.0x. Existing row unchanged.
9	Flow-Scaled sizing loop	<i>index.html updateFlowBadge + szRisk oninput + APPLY</i>	—	B	1	APPLY twice: value stable. _userBaseRisk unchanged.
10	Login flicker	<i>index.html line 4046 vLanding</i>	—	B	1	No sign-in flash on hard refresh while logged in.

Every item starts with: Identity contract decision stated · GitNexus impact run · Failure brainstorm complete · Sandbox (PATH A) or code review (PATH B) output shown · Diff shown → commit gate → push gate.